Experiment No: 09

Student Name: Amol Saurav
UID: 23BCS10067
Branch: BE-CSE
Section/Group: KRG_2B
Semester: 6th
Date of Performance: 15/04/2026
Subject Code: 23CSH-314
Subject Name: System Design

Aim:

To design and analyze a scalable movie ticket booking system that enables users to search movies, view shows, book tickets, and complete payments while ensuring high availability, low latency, and strong consistency during seat booking at large scale.

Objective:

- To understand system design principles for large-scale movie booking systems.
- To design APIs for users, movies, shows, bookings, and payments.
- To identify functional and non-functional requirements.
- To apply distributed system concepts such as CAP theorem (especially consistency in seat booking).
- To design a highly available and scalable system architecture capable of handling high traffic (e.g., 100M DAU).

Procedure:

- Identify functional requirements of the movie booking system such as user registration, movie search, ticket booking, and payment processing.
- Define non-functional requirements such as scalability (100M DAU), high availability, low latency, and strong consistency for seat booking.
- Analyze CAP theorem trade-offs, prioritizing **consistency** during concurrent seat booking to prevent double booking.
- Identify core entities such as users, movies, theatres, shows, seats, bookings, and payments.
- Design APIs for key functionalities including authentication, movie listing/search, seat selection, booking, and payment.
- Design the system architecture using tools like Draw.io, incorporating components like load balancers, microservices, caching (Redis), and databases.
- Validate the design by simulating real-world scenarios such as multiple users attempting to book the same seat simultaneously.

Functional Requirements:

- User should be able to register in the system.
- Users should be able to search for any movie.
- Users should be able to book the ticket of the movie.
- Users should be able to complete the payment.
- Users should be able to view all the movies available.

Core Entities of the System:

- Users
- Movie
- Seats
- Booking
- Payment

API Design:

Users

- POST: /auth/signup
- POST: /auth/login

Movie

- GET /api/v1/movies
- GET /api/v1/movies/search
- GET /api/v1/movies/{movie_id}/shows

Seats

- GET /api/v1/shows/{show_id}/seats
- POST /api/v1/bookings/lock
- GET: /api/v1/funds/history

Booking

- POST /api/v1/bookings/confirm
- GET /api/v1/users/{user_id}/bookings

Payment

- POST /api/v1/payments/verify

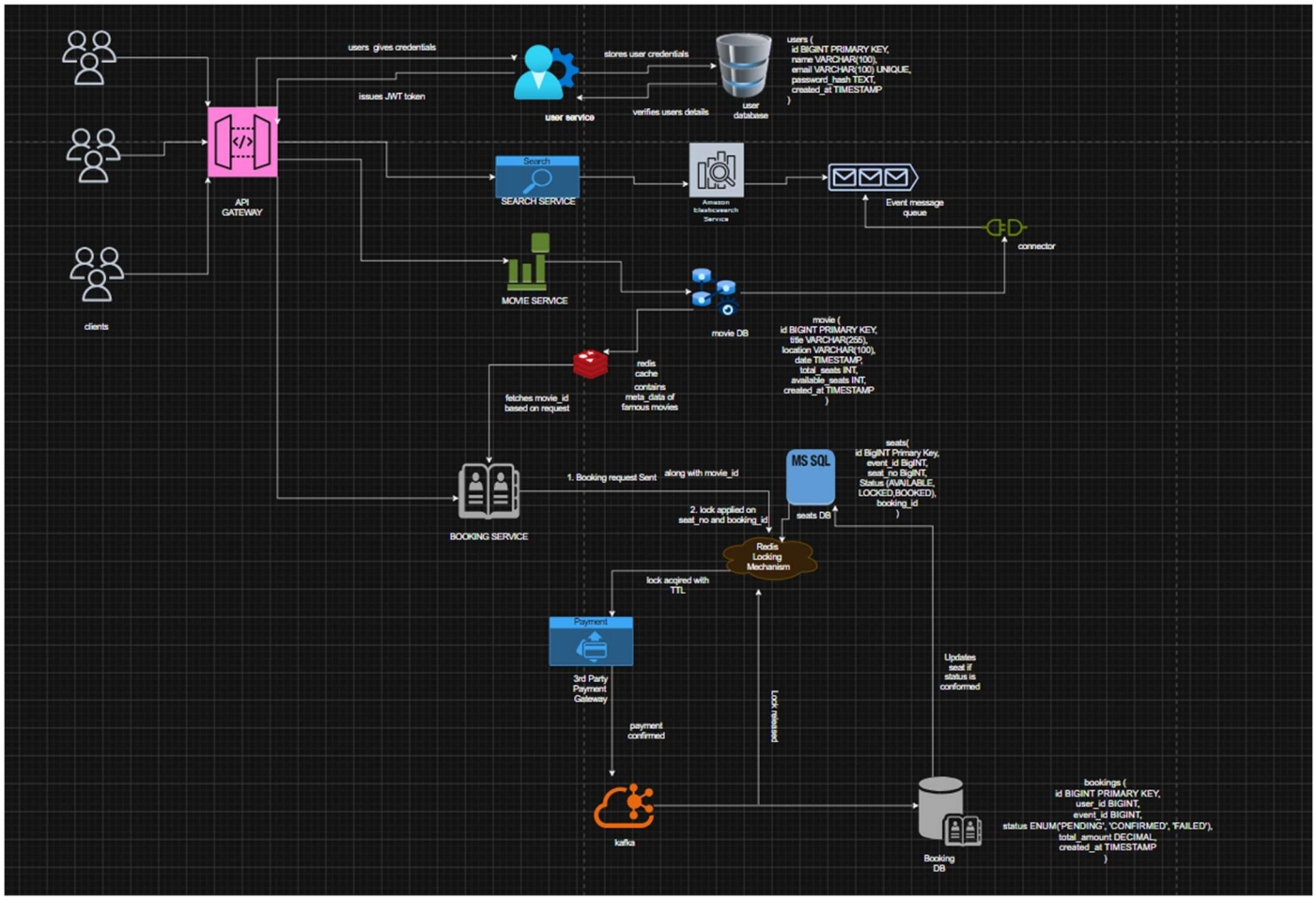
Non-Functional Requirements:

- System should be able to handle high traffic.
- Target scale: 100 million daily active users (DAU).
- System should be available when searching for any movie.
- System must be highly consistent while booking tickets.
- When two users are booking a ticket, they should never be able to book the exact same seat at the exact same time.

High Level Design: -

The system consists of client applications connected through an API Gateway that handles authentication (JWT), routing, and request management, and is supported by core microservices including User Service, Search Service, Movie Service, Booking Service, and Payment Service. The User Service manages user data, the Search Service enables fast movie discovery using a search engine, and the Movie Service handles movie-related data with Redis caching for low-latency access. The Booking Service manages seat selection and ensures strong consistency using a Redis-based locking mechanism to prevent double booking, with seat states maintained as available, locked, or booked in a dedicated database. The Payment Service integrates with a third-party gateway to process

transactions and finalize bookings. An event-driven architecture using a message queue (such as Kafka) enables asynchronous communication, while separate databases for each service ensure scalability, reliability, and high availability.



Outcome:

The movie booking system was successfully designed with key features such as user registration, movie search, viewing available shows, ticket booking, and payment processing, while ensuring scalability, low latency, high availability, and strong consistency to prevent double booking in real-world scenarios.